Exercice 6.1

L'interface `TranScalable` exprime qu'une figure est à la fois `Translatable` et `Scalable`. Le programme ci-dessous manipule des `Rectangle` et des `Circle`.

```
public static void main(String[] args) {
    List<TranScalable> ls = new ArrayList<>();
    // Coin superieur gauche en (1, 2), largeur 3, hauteur 4
    ls.add(new Rectangle(1, 2, 3, 4));
    // Centre en (2, 1), rayon 4
    ls.add(new Circle(2, 1, 4));
    ls.add(new Rectangle(6, 3, 2, 2));
    System.out.println(ls);
    for(TranScalable forme: ls)
        forme.translate(1, 2);
    System.out.println(ls);
    for(TranScalable forme: ls)
        forme.scale(2);
    System.out.println(ls);
}
```

Il produit l'output suivant :

```
[<Rectangle en (1.0, 2.0) <-> 3.0 | 4.0>, <Cercle (2.0, 1.0) <-> 4.0>, <Rectangle en (6.0, 3.0) <-> 2.0 | 2.0>]
[<Rectangle en (2.0, 4.0) <-> 3.0 | 4.0>, <Cercle (3.0, 3.0) <-> 4.0>, <Rectangle en (7.0, 5.0) <-> 2.0 | 2.0>]
[<Rectangle en (2.0, 4.0) <-> 6.0 | 8.0>, <Cercle (3.0, 3.0) <-> 8.0>, <Rectangle en (7.0, 5.0) <-> 4.0 | 4.0>]
```

Définissez les interfaces voulues et les classes `Point`, `Rectangle` et `Circle` pour compléter le programme. Un rectangle sera défini par un `Point` (coin supérieur gauche), sa largeur et sa hauteur. Un cercle sera défini par un `Point` (son centre) et son rayon.

Si du code semble dupliqué, pensez aux méthodes par défaut des interfaces.

Faites quelques essais pertinents à l'aide de tests unitaires.

Exercice 6.2

Ecrivez deux `Comparator<Person>`. Le premier compare les personnes selon leur nom de famille et le second selon leur prénom.

A l'aide de tests unitaires, triez (à l'aide de `Collections.sort()` ou de la méthode `sort()` de l'interface `List`) une liste de personnes à l'aide du second puis du premier. Faites des essais pour constater que le tri est stable (l'ordre relatif des éléments de même rang est préservé).

Exercice 6.3

Utilisez un `TreeSet<Person>` de façon à maintenir l'ordre dans un carnet d'adresse composé de `Person`. Sans modifier la classe `Person`, arrangez vous pour que les personnes soient triées par ordre de nom, prénom et âge. Constatez également qu'il n'y a pas de doublon dans un `TreeSet<E>`.